

RoBERTa Fine-Tuning - Line by Line

Part 1: Setup and Data Preparation

Imports

```
1 from transformers import RobertaTokenizer, RobertaForSequenceClassification,
    get_linear_schedule_with_warmup
2 from torch.utils.data import Dataset, DataLoader
3 import torch.nn as nn
4 import torch
```

Loads Hugging Face transformers library for RoBERTa model and PyTorch utilities.

Tokenizer Loading

```
1 tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
```

Downloads and loads pretrained RoBERTa tokenizer with its vocabulary.

Dataset Class Definition

```
1 class SentimentRobertaDataset(Dataset):
```

Creates custom dataset class for RoBERTa input format.

```
1     def __init__(self, texts, labels, tokenizer, max_length=128):
2         self.texts = texts
3         self.labels = labels
4         self.tokenizer = tokenizer
5         self.max_length = max_length
```

Stores texts, labels, tokenizer, and maximum sequence length (128 tokens).

```
1     def __len__(self):
2         return len(self.texts)
```

Returns dataset size.

```
1     def __getitem__(self, idx):
2         text = str(self.texts[idx])
3         label = self.labels[idx]
```

Retrieves text and label at given index, converts text to string.

```
1         encoding = self.tokenizer(
2             text,
3             add_special_tokens=True,
4             max_length=self.max_length,
5             padding='max_length',
```

```

6         truncation=True,
7         return_attention_mask=True,
8         return_tensors='pt'
9     )

```

Tokenizes text with:

- `add_special_tokens=True`: Adds [CLS] and [SEP] tokens
- `max_length=128`: Truncates at 128 tokens
- `padding='max_length'`: Pads to 128 tokens
- `truncation=True`: Cuts off excess tokens
- `return_attention_mask=True`: Creates mask for real vs padding tokens
- `return_tensors='pt'`: Returns PyTorch tensors

```

1     return {
2         'input_ids': encoding['input_ids'].flatten(),
3         'attention_mask': encoding['attention_mask'].flatten(),
4         'label': torch.tensor(label, dtype=torch.long)
5     }

```

Returns dictionary with flattened token IDs, attention mask, and label.

Dataset Creation

```

1 print("      Creation des datasets RoBERTa avec TEXTE BRUT...")
2 train_dataset = SentimentRobertaDataset(
3     texts=train_df['text'].tolist(),
4     labels=train_df['label'].tolist(),
5     tokenizer=tokenizer,
6     max_length=128
7 )

```

Creates training dataset using raw text (not preprocessed - preserves emojis, punctuation, capitalization).

```

1 test_dataset = SentimentRobertaDataset(
2     texts=test_df['text'].tolist(),
3     labels=test_df['label'].tolist(),
4     tokenizer=tokenizer,
5     max_length=128
6 )

```

Creates test dataset.

DataLoader Creation

```

1 train_loader_roberta = DataLoader(train_dataset, batch_size=32, shuffle=True
2     , num_workers=0)
3 test_loader_roberta = DataLoader(test_dataset, batch_size=32, shuffle=False,
4     num_workers=0)

```

Creates data loaders with batch size 32. Training data shuffled, test data in order.

```

1 print(f"    Datasets crees :")
2 print(f"    - Train : {len(train_dataset)} echantillons")
3 print(f"    - Test  : {len(test_dataset)} echantillons")

```

Prints dataset sizes.

Model Loading

```

1 model = RobertaForSequenceClassification.from_pretrained(
2     'roberta-base',
3     num_labels=3,
4     problem_type="single_label_classification"
5 ).to(device)

```

Downloads pretrained RoBERTa-base model and adds classification head for 3 labels. Moves to GPU/CPU.

```

1 print(f"\n Modele RoBERTa-base charge ({sum(p.numel() for p in model.
    parameters())/1e6:.1f}M parametres)")

```

Prints total parameters in millions.

Part 2: Training Setup and Execution

Imports

```

1 from torch.optim import AdamW
2 from sklearn.metrics import f1_score, accuracy_score, classification_report
3 import time

```

Loads optimizer, evaluation metrics, and timing utilities.

Optimizer Setup

```

1 optimizer = AdamW(model.parameters(), lr=2e-5, eps=1e-8)

```

Creates AdamW optimizer with learning rate 2e-5 (standard for fine-tuning) and epsilon 1e-8 for numerical stability.

```

1 total_steps = len(train_loader_roberta) * 4

```

Calculates total training steps: batches per epoch $\times$ 4 epochs.

```

1 scheduler = get_linear_schedule_with_warmup(
2     optimizer,
3     num_warmup_steps=0,
4     num_training_steps=total_steps
5 )

```

Creates learning rate scheduler that linearly decreases from initial LR to 0. No warmup steps.

Training Function

```
1 def train_epoch_roberta(model, dataloader, optimizer, scheduler, device):  
2     model.train()
```

Defines training function and sets model to training mode.

```
1     total_loss = 0  
2     all_preds = []  
3     all_labels = []
```

Initializes accumulators for loss, predictions, and labels.

```
1     for batch in dataloader:  
2         input_ids = batch['input_ids'].to(device)  
3         attention_mask = batch['attention_mask'].to(device)  
4         labels = batch['label'].to(device)
```

Iterates through batches and moves data to device.

```
1     model.zero_grad()
```

Clears gradients from previous step.

```
1         outputs = model(  
2             input_ids=input_ids,  
3             attention_mask=attention_mask,  
4             labels=labels  
5         )
```

Forward pass through RoBERTa. Automatically computes loss when labels provided.

```
1         loss = outputs.loss  
2         logits = outputs.logits
```

Extracts loss and logits (predictions) from output object.

```
1         loss.backward()
```

Computes gradients via backpropagation.

```
1         torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

Clips gradient norms to maximum of 1.0 to prevent exploding gradients.

```
1         optimizer.step()  
2         scheduler.step()
```

Updates model parameters and adjusts learning rate.

```
1         total_loss += loss.item()  
2         preds = logits.argmax(dim=1)  
3         all_preds.extend(preds.cpu().numpy())  
4         all_labels.extend(labels.cpu().numpy())
```

Accumulates loss, gets predicted class (highest logit), stores predictions and labels.

```
1     avg_loss = total_loss / len(dataloader)  
2     f1 = f1_score(all_labels, all_preds, average='macro')  
3     return avg_loss, f1
```

Computes average loss and macro F1 score across all classes.

Evaluation Function

```
1 def evaluate_roberta(model, dataloader, device):
2     model.eval()
```

Sets model to evaluation mode (disables dropout).

```
1     total_loss = 0
2     all_preds = []
3     all_labels = []
4
5     with torch.no_grad():
```

Disables gradient computation for efficiency.

```
1         for batch in dataloader:
2             input_ids = batch['input_ids'].to(device)
3             attention_mask = batch['attention_mask'].to(device)
4             labels = batch['label'].to(device)
5
6             outputs = model(
7                 input_ids=input_ids,
8                 attention_mask=attention_mask,
9                 labels=labels
10            )
```

Forward pass without gradient tracking.

```
1             loss = outputs.loss
2             logits = outputs.logits
3
4             total_loss += loss.item()
5             preds = logits.argmax(dim=1)
6             all_preds.extend(preds.cpu().numpy())
7             all_labels.extend(labels.cpu().numpy())
```

Accumulates results.

```
1     avg_loss = total_loss / len(dataloader)
2     f1_macro = f1_score(all_labels, all_preds, average='macro')
3     f1_weighted = f1_score(all_labels, all_preds, average='weighted')
4     acc = accuracy_score(all_labels, all_preds)
5     return avg_loss, f1_macro, f1_weighted, acc, all_preds, all_labels
```

Computes multiple metrics: loss, macro F1, weighted F1, accuracy.

Training Loop

```
1 num_epochs = 4
2 best_f1 = 0
```

Sets number of epochs and initializes best F1 tracker.

```
1 print("\n" + "="*60)
2 print("DEBUT DU FINE-TUNING - ROBERTA-BASE")
3 print("="*60)
4 print(f"          Configuration : LR=2e-5 | Batch=32 | Epochs={num_epochs}")
5 print(f"  Texte BRUT utilise (pas de pretraitement custom)\n")
```

Prints training configuration.

```
1 for epoch in range(num_epochs):
2     start_time = time.time()
```

Starts epoch timing.

```
1 train_loss, train_f1 = train_epoch_roberta(  
2     model, train_loader_roberta, optimizer, scheduler, device  
3 )
```

Trains for one epoch.

```
1 val_loss, val_f1_macro, val_f1_weighted, val_acc, _, _ =  
2     evaluate_roberta(  
3         model, test_loader_roberta, device  
4     )
```

Evaluates on test set.

```
1 elapsed = time.time() - start_time  
2  
3 print(f"Epoch {epoch+1:2d}/{num_epochs} | "  
4       f"Train Loss: {train_loss:.4f} | Train F1: {train_f1:.4f} | "  
5       f"Val Loss: {val_loss:.4f} | Val F1 (macro): {val_f1_macro:.4f} | "  
6       f"Val Acc: {val_acc:.4f} | Time: {elapsed:.0f}s")
```

Prints epoch results.

```
1 if val_f1_macro > best_f1:  
2     best_f1 = val_f1_macro  
3     torch.save(model.state_dict(), 'best_model_roberta.pth')  
4     print(f"Meilleur modele sauvegarde (F1 macro: {best_f1:.4f})")
```

Saves model if validation F1 improves.

Final Evaluation

```
1 model.load_state_dict(torch.load('best_model_roberta.pth'))  
2 _, f1_macro, f1_weighted, acc, preds, labels = evaluate_roberta(model,  
3     test_loader_roberta, device)
```

Loads best model and evaluates.

```
1 print("\n" + "="*60)  
2 print("RESULTATS FINAUX - ROBERTA-BASE")  
3 print("="*60)  
4 print(f"Accuracy      : {acc:.4f}")  
5 print(f"F1 Macro       : {f1_macro:.4f}")  
6 print(f"F1 Weighted    : {f1_weighted:.4f}")  
7 print("\nClassification Report:")  
8 print(classification_report(  
9     labels,  
10    preds,  
11    target_names=['Negatif', 'Neutre', 'Positif'],  
12    digits=4  
13 ))
```

Prints detailed results and per-class metrics.

```
1 test_df_sample = test_df.copy().reset_index(drop=True)  
2 test_df_sample['pred_roberta'] = preds[:len(test_df_sample)]  
3 test_df_sample.to_csv('test_predictions_roberta.csv', index=False)  
4 print(f"\n    Predictions sauvegardees : 'test_predictions_roberta.csv'")
```

Saves predictions to CSV file.

```

1 print("\n" + "="*60)
2 print("          COMPARAISON FINALE DES PERFORMANCES")
3 print("="*60)
4 print(f"{'Architecture':<40} {'F1 Macro':<12} {'      vs Baseline':<15}")
5 print("-"*60)
6 print(f"{'LSTM + Attention (baseline)':<40} {0.7069:<12.4f} {'      ':<15}")
7 print(f"{'BiLSTM + Attention':<40} {0.7151:<12.4f} {'+0.82 pts':<15}")
8 print(f"{'Hybride CNN+BiLSTM+Att':<40} {0.7148:<12.4f} {'+0.79 pts':<15}")
9 print(f"{'CNN 1D seul':<40} {0.6839:<12.4f} {'-2.30 pts':<15}")
10 print(f"{'RoBERTa-base (pre-entraîne)':<40} {f1_macro:<12.4f} {f'+{f1_macro} - 0.7069:.2f pts':<15}")
11 print("="*60)
12 print(f"\n          Gain RoBERTa vs meilleur custom : +{f1_macro - 0.7151:.2f} points")
13 print(f"          Ce gain provient de l'exploitation des signaux bruts (emojis , '!!!', majuscules)")
14 print(f"          que notre pretraitement custom éliminait involontairement.")

```

Compares RoBERTa performance against custom models.

Tools Used

Hugging Face Transformers

- RobertaTokenizer: Pretrained tokenizer with vocabulary
- RobertaForSequenceClassification: Pretrained model with classification head
- get_linear_schedule_with_warmup: Learning rate scheduler

PyTorch

- torch.optim.AdamW: Optimizer with weight decay
- torch.utils.data.Dataset: Base dataset class
- torch.utils.data.DataLoader: Batch data loading
- torch.nn.utils.clip_grad_norm_: Gradient clipping
- torch.save, torch.load: Model checkpoint management

Scikit-learn

- f1_score: F1 metric with macro/weighted averaging
- accuracy_score: Overall accuracy
- classification_report: Detailed per-class metrics

Key Concepts

- **Fine-tuning:** Adapting pretrained model to specific task
- **Raw text:** No preprocessing preserves emojis, punctuation, capitalization
- **Gradient clipping:** Prevents training instability
- **Learning rate scheduling:** Gradual LR decrease improves convergence
- **Model checkpointing:** Saves best performing version
- **Macro F1:** Unweighted average across classes (better for imbalanced data)